

Relazione per Programmazione a Oggetti

HearthCode

Massimiliano Ria
Simone Marsili
Francesco Dama
Lorenzo Mercuriali

Indice

1	Analisi	1
1.1	Descrizione e requisiti	1
1.2	Modello del Dominio	3
2	Design	5
2.1	Architettura	5
2.2	Design Dettagliato	6
2.2.1	Massimiliano Ria	6
2.2.2	Simone Marsili	11
2.2.3	Francesco Dama	12
2.2.4	Lorenzo Mercuriali	12
3	Sviluppo	14
3.1	Testing automatizzato	14
3.2	Note di sviluppo	14
3.2.1	Massimiliano Ria	14
3.2.2	Simone Marsili	14
3.2.3	Francesco Dama	14
3.2.4	Lorenzo Mercuriali	14
4	Commenti Finali	15
4.1	Autovalutazione e lavori futuri	15
4.1.1	Massimiliano Ria	15
4.1.2	Simone Marsili	15
4.1.3	Francesco Dama	15
4.1.4	Lorenzo Mercuriali	15
4.2	Opzionale - Difficoltà incontrate e commenti per i docenti . . .	15
A	Guida Utente	16
B	Esercitazioni di Laboratorio	17

1. Analisi

1.1 Descrizione e requisiti

Il software HearthCode ha l'obiettivo di realizzare una versione semplificata di un gioco di carte strategico a turni ispirato a Hearthstone. L'applicazione appartiene al dominio dei giochi in cui due partecipanti si affrontano utilizzando risorse limitate, carte con caratteristiche specifiche e un insieme di regole che stabilisce quali azioni possono essere eseguite in ogni momento della partita.

A ogni partita partecipano due contendenti: il giocatore umano e un avversario controllato automaticamente dal sistema. Entrambi dispongono di un ammontare di punti vita, di una riserva di mana, di un mazzo da cui pescare e di una mano contenente le carte disponibili. All'inizio di ogni turno il sistema esegue automaticamente le operazioni obbligatorie di preparazione: aggiorna e rifornisce il mana del giocatore attivo e pesca una carta dal suo mazzo. Se la mano ha raggiunto la capienza massima, la carta pescata non viene aggiunta alla mano e viene invece bruciata. In seguito il giocatore può schierare creature dalla mano al campo di gioco o utilizzare quelle già attive per attaccare il giocatore nemico, danneggiando le sue creature o i suoi punti vita.

Una carta può essere giocata solo se il giocatore possiede una quantità di mana almeno pari al suo costo e se il campo non ha già raggiunto il numero massimo di creature consentite. Una creatura appena schierata entra in gioco in stato inattivo e può attaccare solo a partire dal turno successivo del suo proprietario. Dopo avere attaccato, la creatura viene esaurita e non può compiere ulteriori attacchi nello stesso turno.

Per evitare situazioni di stallo, il gioco prevede una penalità nel caso in cui un giocatore debba pescare da un mazzo ormai esaurito. Poiché i mazzi hanno la stessa dimensione e il pescaggio è automatico e obbligatorio per entrambi, l'esaurimento avviene in modo bilanciato nello stesso ciclo di turni. Da quel momento, ogni tentativo di pescaggio a mazzo vuoto danneggia i

punti vita dell'eroe con valori progressivamente crescenti, garantendo che la partita arrivi sempre a una conclusione.

Il sistema e' responsabile della gestione dello stato della partita e dell'applicazione coerente delle regole. Di conseguenza, deve impedire mosse non valide, come giocare una carta non presente nella mano, usare una creatura non ancora attiva, attaccare con una creatura gia' utilizzata o giocare una carta senza mana sufficiente. La partita prosegue attraverso l'alternanza dei turni fino al verificarsi di una condizione di conclusione: quando i punti vita di un giocatore scendono a zero, l'avversario viene dichiarato vincitore.

Requisiti funzionali

- Il sistema deve inizializzare una partita tra il giocatore umano e l'avversario controllato automaticamente, assegnando a entrambi punti vita, mazzo, mana iniziale e risorse di gioco.
- Il sistema deve gestire l'alternanza dei turni, aggiornando correttamente il giocatore attivo, il mana disponibile, il pescaggio obbligatorio e lo stato di attivazione delle creature schierate.
- Il sistema deve gestire il pescaggio automatico e permettere al giocatore di giocare carte rispettando le regole definite, in particolare il costo in mana, la presenza della carta nella mano e lo spazio disponibile sul campo.
- Il sistema deve bruciare la carta pescata quando la mano del giocatore ha raggiunto la capienza massima.
- Il sistema deve applicare una penalita' incrementale ai punti vita del giocatore quando il suo mazzo e' esaurito e deve comunque pescare una carta.
- Il sistema deve permettere alle creature attive di attaccare creature avversarie o il giocatore nemico, aggiornando punti vita, stato delle creature ed eventuale rimozione delle creature sconfitte.
- Il sistema deve impedire l'esecuzione di mosse illecite, segnalando l'errore senza compromettere lo stato della partita.
- Il sistema deve gestire autonomamente il turno dell'avversario, scegliendo ed eseguendo azioni valide sulla base dello stato corrente della partita.
- Il sistema deve determinare correttamente la conclusione della partita e il vincitore quando uno dei due giocatori esaurisce i propri punti vita.

Requisiti non funzionali

- Il sistema deve essere portatile: il comportamento dell'applicazione deve rimanere indipendente dall'hardware e dal sistema operativo utilizzati.
- L'applicazione deve rispondere in tempi rapidi alle azioni dell'utente e ridurre i ritardi percepibili, in modo da garantire un'esperienza di gioco fluida.
- Il sistema deve mantenere uno stato di gioco consistente anche in presenza di input non validi, evitando che errori di interazione producano situazioni incoerenti.

1.2 Modello del Dominio

Il sistema si basa su un tavolo di gioco, rappresentato dal *board game*, che costituisce lo spazio condiviso della partita e coordina lo stato dei due partecipanti. Il tavolo mantiene il riferimento ai giocatori, alle rispettive armate e al turno corrente; inoltre applica le regole principali della partita, come il posizionamento delle carte, la gestione degli attacchi, il cambio turno e la verifica delle condizioni di vittoria.

Le carte schierate sul campo sono organizzate in due armate, una per ciascun giocatore. L'armata rappresenta l'insieme delle creature controllate da un partecipante e distingue le creature attive da quelle appena giocate, che devono attendere il turno successivo prima di poter attaccare. L'armata e' inoltre responsabile di verificare se una creatura puo' agire, di disabilitarla dopo un attacco e di rimuoverla dal campo quando i suoi punti vita vengono azzerati.

Ogni giocatore e' caratterizzato da un identificatore, dai propri punti vita, dalla quantita' di mana disponibile, da una mano e da un mazzo. La mano contiene le carte che il giocatore puo' scegliere di giocare durante il turno, mentre il mazzo rappresenta l'insieme delle carte non ancora pescate. La pesca trasferisce una carta dal mazzo alla mano quando possibile; se il mazzo e' esaurito, il sistema applica una penalita' ai punti vita del giocatore secondo le regole previste.

Nella versione base del software, ogni carta giocabile e' modellata come una creatura. Una creatura e' definita da un nome, un costo in mana, un valore di attacco e un numero di punti vita. Il costo in mana determina se la carta puo' essere giocata, il valore di attacco indica il danno inflitto durante

un combattimento e i punti vita determinano la permanenza della creatura sul campo.

L'avversario automatico utilizza una rappresentazione dello stato della partita per generare le azioni disponibili, valutarne la convenienza ed eseguire una sequenza di mosse valide durante il proprio turno. In questo modo il comportamento dell'avversario rimane vincolato alle stesse regole applicate al giocatore umano.

2. Design

2.1 Architettura

L'architettura dell'applicazione segue un'impostazione riconducibile al pattern *Model-View-Controller* (MVC). Il *Model* e' rappresentato dall'interfaccia **BoardGame** e dalla sua implementazione **BoardGameImpl**, che costituiscono il punto d'ingresso della logica di partita. Il modello incapsula lo stato del gioco, le operazioni principali della partita e la gestione del turno, delegata internamente al **TurnManager**. Inoltre espone metodi di controllo e interrogazione dello stato senza dipendere dalla View o dalle modalita' con cui l'utente interagisce con l'applicazione.

Il Controller principale e' la classe **MainControllerImpl**. Essa implementa le interfacce **MainController** e **SceneCoordinator**, coordina il flusso applicativo generale, crea le schermate, registra le scene nella vista principale e gestisce la navigazione tra menu, selezione della difficulta', database delle carte, partita ed esito finale.

Per evitare un'eccessiva concentrazione di responsabilita', il controller principale e' affiancato da controller specializzati per le singole schermate:

- **MenuController**
- **DifficultySelectionController**
- **DatabaseController**
- **MatchController**
- **EndMatchController**

Tra questi, **MatchController** riceve le azioni della schermata di partita, interagisce con **BoardGame** e integra i servizi dedicati al turno dell'intelligenza artificiale.

La View e' rappresentata dall'interfaccia **MainView**, implementata da **MainViewImpl**. Essa funge da finestra principale dell'applicazione, mantiene le

scene registrate e visualizza una sola `Scene` alla volta tramite un `CardLayout`. Le schermate concrete sono:

- `MenuScene`
- `DifficultySelectionScene`
- `DatabaseScene`
- `MatchScene`
- `EndMatchScene`

Ciascuna espone un'interfaccia specifica per registrare le azioni utente. I controller collegano tali azioni alla logica applicativa tramite callback, mantenendo separata la gestione degli eventi grafici dalla logica di dominio.

Durante la partita, il flusso principale degli input segue quindi la catena `View-Controller-Model`: la scena intercetta l'interazione dell'utente, il controller associato la interpreta e invoca le operazioni del modello. Gli aggiornamenti dello stato di partita vengono invece propagati dal modello alla schermata tramite il contratto `GameObserver`, implementato da `MatchView`. La navigazione tra schermate avviene attraverso `SceneCoordinator`, che usa `MainView` per rendere effettivo il cambio di scena.

Questa organizzazione garantisce un buon disaccoppiamento tra modello, vista e controller. In particolare, la sostituzione della `View` non richiede modifiche al modello e richiede soltanto che la nuova interfaccia grafica rispetti i contratti esposti da `MainView`, `Scene` e dalle interfacce di vista specifiche. La logica di dominio e il coordinamento applicativo restano quindi indipendenti dalla tecnologia usata per rappresentare l'interfaccia utente.

2.2 Design Dettagliato

2.2.1 Massimiliano Ria

Adattamento dell'interfaccia a diverse risoluzioni

Problema: Dopo aver ottenuto un'interfaccia grafica funzionante e un'architettura coerente, è emersa la necessità di mantenerne l'aspetto uniforme su risoluzioni e rapporti d'aspetto diversi.

Soluzione: Un primo approccio al problema è stato l'introduzione della classe `ViewMetrics`, usata come punto di accesso unico alle principali dimensioni

grafiche dell'applicazione. Nella sua versione iniziale, tali misure venivano calcolate a partire dalla dimensione fisica dello schermo. La classe determina una sola volta le grandezze necessarie e le espone tramite metodi statici dedicati, come ad esempio `cardWidth()` e `cardHeight()`. In questo modo si riduce la duplicazione del codice e ogni scena può richiedere solo la metrica di cui ha bisogno.

I test successivi hanno però evidenziato un limite: scalare ogni componente in base all'intero schermo non garantiva automaticamente una buona visualizzazione. Su schermi con risoluzioni elevate o con un rapporto d'aspetto diverso da quello previsto, alcuni elementi risultavano troppo grandi o disposti in modo poco bilanciato. Inoltre lo sfondo del menu, realizzato nativamente con risoluzione 3840x2160 e rapporto 16:9, rischiava di essere adattato a superfici non coerenti con la sua proporzione originale, producendo tagli dell'immagine o distorsioni impreviste.

Per risolvere il problema è stato introdotto il concetto di *viewport*: l'applicazione continua ad aprirsi a schermo intero, ma la zona effettiva in cui vengono disegnate le scene corrisponde alla più grande area 16:9 contenibile nello schermo dell'utente. Il calcolo è centralizzato nel metodo `ViewMetrics.viewportSize(Dimension)`, che confronta il rapporto tra la larghezza e l'altezza disponibili con il rapporto obiettivo 16:9. Se lo schermo è più largo del necessario, viene mantenuta l'altezza e ridotta la larghezza; se invece è troppo alto, viene mantenuta la larghezza e ridotta l'altezza.

La classe `BlackBounds` applica concretamente questa scelta: viene usata da `MainViewImpl` come pannello radice della finestra e contiene il pannello delle scene gestito dal `CardLayout`. Nell'override del metodo `doLayout()`, richiamato automaticamente da Swing quando la finestra viene mostrata per la prima volta o quando cambia dimensione, `BlackBounds` calcola la viewport a partire dalla dimensione corrente della finestra, centra l'unico componente figlio e gli assegna l'area 16:9 risultante. Tutto lo spazio esterno rimane nero, generando bande laterali oppure superiori e inferiori in base alla situazione.

Multi-Risoluzione per schermi Hi-DPI

Problema: Nelle ultime fasi di sviluppo è emerso un bug legato alla qualità delle immagini delle carte. Il problema non si presentava in modo uniforme: su alcuni schermi le texture risultavano nitide, mentre su altri apparivano sgranate o sfocate, senza che inizialmente fosse riconoscibile un legame evidente con la risoluzione nominale del monitor.

L'analisi successiva ha ricondotto il comportamento agli schermi Hi-DPI. Su questi dispositivi il sistema operativo può applicare uno *zoom* logico: l'interfaccia continua a ragionare in pixel logici, ma ogni pixel logico viene

poi rappresentato tramite più pixel fisici. Questa scelta evita che finestre e componenti diventino troppo piccoli su pannelli molto densi, ma introduce una differenza tra la dimensione richiesta da Swing e la dimensione realmente usata dal dispositivo di output. Nel caso specifico, le classi della View ottenevano le immagini tramite `ImageLoader`, che le restituiva già scalate alle dimensioni calcolate da `ViewMetrics`. Quando però Swing disegnava quelle icone su uno schermo con fattore di scala maggiore di 1, l'immagine già ridotta veniva adattata a una superficie fisica più grande, producendo l'effetto di sgranatura osservato sulle carte.

Soluzione: L'idea adottata è stata quella di non trattare più una richiesta di immagine scalata come una singola bitmap, ma come un'immagine multi-risoluzione. Per ogni risorsa grafica richiesta a una certa dimensione logica viene preparata una variante coerente con quella dimensione e, quando necessario, una seconda variante coerente con la dimensione fisica del dispositivo. In questo modo le classi della View continuano a chiedere immagini in termini di pixel logici, ma il sottosistema grafico di Java può selezionare la variante più adatta al momento del disegno.

Il punto centrale del fix è `AsyncCachedImageRepository`, che implementa `ImageRepository` ed è usata da `ImageLoaderProxy` dietro alla facciata statica `ImageLoader`. Quando riceve una `ImageLoadRequest` scalata, il repository calcola prima la dimensione fisica equivalente tramite il metodo `deviceSize(int, int)`. Questo metodo legge il `defaultTransform` della configurazione grafica principale, ottenuta da `GraphicsEnvironment`, e moltiplica larghezza e altezza logiche per i fattori di scala dello schermo. Nei contesti senza ambiente grafico viene invece usato un fattore 1.0, così il caricamento resta valido anche in esecuzioni headless.

Una volta note le due dimensioni, `AsyncCachedImageRepository` costruisce l'icona nel metodo `createScaledIcon`. La risorsa originale viene letta con `ImageIO` come `BufferedImage`, così può essere ridimensionata in modo controllato. Se dimensione logica e dimensione fisica coincidono, viene restituita una normale `ImageIcon`. Se invece lo schermo richiede più pixel fisici, viene creata una `BaseMultiResolutionImage` contenente la variante logica e quella fisica, poi incapsulata in una `ImageIcon`. Le chiavi della cache delle immagini scalate includono sia la dimensione logica sia quella fisica, evitando di riutilizzare per errore la stessa bitmap su schermi con fattori di scala diversi.

Il ridimensionamento non usa più direttamente `getScaledInstance`, ma una procedura esplicita basata su `Graphics2D`. Il metodo `scaleImage` riceve l'immagine sorgente e la dimensione obiettivo, poi la riduce con passi intermedi: finché la distanza dal risultato finale resta elevata, la bitmap viene

dimezzata; solo alla fine viene prodotta la dimensione esatta. Ogni passaggio è disegnato da `renderScaledImage` con interpolazione bicubica, qualità di rendering elevata e antialiasing. Questa scelta aumenta leggermente il costo del precaricamento, ma concentra il lavoro in un punto controllato e produce una qualità più stabile.

Le classi che mostrano effettivamente le carte non devono conoscere il dettaglio Hi-DPI. `CardComponent` continua a caricare fronte e retro tramite `ImageLoader`. Le misure restano quelle di `ViewMetrics`; nel metodo `paintComponent` il componente disegna l'immagine ricevuta nello spazio del bottone. Analogamente `CardsPanel` usa lo stesso caricatore per costruire gli `IconPanel` del database. Il catalogo `ImagePreloadCatalog` produce invece le richieste scalate per carte, bottoni e banner, così `ImageLoader` può precaricare le varianti prima della visualizzazione delle scene.

Prima di stabilizzare questa soluzione sono stati valutati anche refactor alternativi. In particolare è stato provato un approccio in cui l'immagine ad alta risoluzione veniva mantenuta più a lungo e ridimensionata nel punto più vicino possibile al disegno finale. La qualità visiva era promettente, ma il compromesso risultava meno adatto all'applicazione: il costo dello scaling tendeva a spostarsi sul primo rendering dei componenti, oppure rendeva il precaricamento più pesante e meno prevedibile nelle schermate con molte carte. La soluzione multi-risoluzione centralizzata nel repository è quindi risultata più equilibrata, perché mantiene l'API della View invariata, conserva il precaricamento asincrono già presente e delega a Java la scelta della variante grafica corretta per lo schermo in uso.

Stato Simulabile per l'IA

Problema: L'approccio scelto per l'implementazione dell'IA in fase di progettazione ha da subito evidenziato un concetto inevitabile: per permettere all'IA di valutare l'applicazione di determinate mosse sullo stato della partita, era necessario che queste venissero effettivamente applicate sul tavolo da gioco, ma modificare direttamente lo stato reale del match avrebbe compromesso la partita in corso.

Soluzione: La soluzione più logica è stata dunque introdurre una rappresentazione separata dello stato della partita, ricostruita in un contesto simulato e interamente modificabile. Questo ruolo è affidato all'interfaccia `AiGameState` e alla sua implementazione `AiGameStateImpl`, che espongono le operazioni minime necessarie alla pianificazione: danneggiare un giocatore, consumare mana, giocare una carta, danneggiare o rimuovere una carta dal campo e produrre una copia dello stato corrente.

La costruzione dello stato simulabile è delegata a `AiGameStateFactory`, implementata da `AiGameStateFactoryImpl`. La factory interroga lo stato reale attraverso `BoardGame` e costruisce lo snapshot copiando i dati dello stato reale e usando copie delle carte. Lo stato del giocatore umano viene creato in maniera dedicata per evitare di copiare dati sensibili che dal punto di vista dell'IA non sono accessibili.

Le informazioni dei singoli giocatori sono isolate dietro `PlayerState` e `PlayerStateImpl`. Questa classe mantiene l'identificativo del giocatore, la sua salute, il mana attuale, l'eventuale mano e l'armata in campo, mentre `CardStateImpl` rappresenta gli stati delle carte copiate.

Infine l'interfaccia `AiStateTransition` completa l'implementazione, rappresentando il concetto di transizione da uno stato simulato a un altro. La classe `AiStateTransitionImpl` infatti crea una copia dello stato ricevuto e poi la modifica in base all'azione da applicare: `PlayCardAction` consuma mana e sposta la carta dalla mano all'armata, `AttackHeroAction` riduce la salute dell'eroe avversario, mentre `AttackCardAction` applica i danni a entrambe le creature e rimuove quelle sconfitte. Gli algoritmi decisionali possono così avere un riferimento a questa classe e utilizzarla per esplorare scenari successivi senza dipendere dalle regole concrete di modifica dello stato.

Politica Decisionale Sostituibile per l'IA

Problema: A un certo punto dell'implementazione dell'IA, è sorta la necessità di integrarla con il ciclo reale del turno. L'algoritmo poteva scegliere una sequenza di mosse su uno stato astratto, ma tali decisioni dovevano poi essere applicate alla partita effettivamente giocata dall'utente. Inoltre la selezione della difficoltà richiedeva di poter sostituire la politica decisionale senza modificare il controller della partita o le regole del modello.

Soluzione: La soluzione adottata separa la scelta delle mosse dalla loro esecuzione reale. Il punto di raccordo principale è rappresentato dall'interfaccia `AiTurnService` e dalla classe `AiTurnServiceImpl`. Il servizio riceve il `BoardGame` corrente, usa `AiGameStateFactory` per costruire lo snapshot simulabile e delega la decisione a un oggetto di tipo `AiDecisionAlgorithm`. Il controller della partita non deve quindi conoscere il funzionamento interno dell'IA: richiede soltanto il piano del turno e ottiene una lista ordinata di `AiAction`.

La sostituibilità della politica decisionale è ottenuta applicando il pattern *Strategy*. L'interfaccia `AiDecisionAlgorithm` definisce il contratto comune degli algoritmi, mentre le implementazioni `GreedySequentialAiAlgorithm` e `DepthLimitedLookaheadAiAlgorithm` rappresentano due strategie alternati-

ve. La prima sceglie iterativamente la migliore azione che produce un miglioramento immediato dello stato, la seconda esplora sequenze di azioni fino a una profondita' limitata e seleziona quella con la valutazione euristica migliore. Entrambe usano gli stessi componenti di supporto: `AiActionGeneratorImpl` per generare le azioni legali, `AiStateTransitionImpl` per applicarle allo stato simulato e `HeuristicAiStateEvaluator` per assegnare un punteggio agli scenari.

La scelta concreta della strategia avviene in `MainControllerImpl`, dove una mappa associa `Difficulty.NORMAL` a `GreedySequentialAiAlgorithm` e `Difficulty.HARD` a `DepthLimitedLookaheadAiAlgorithm`. Quando l'utente avvia una partita, il controller costruisce `AiTurnServiceImpl` passando l'algoritmo corrispondente alla difficulta' selezionata. In questo modo l'aggiunta di una nuova difficulta' o di un nuovo algoritmo richiede soltanto una nuova implementazione della strategia e la sua registrazione nel punto di creazione.

L'applicazione delle azioni decise dall'IA e' invece responsabilita' di `AiActionExecutor`, implementata da `AiActionExecutorImpl`. Questa classe traduce le azioni astratte prodotte dagli algoritmi nelle operazioni effettive di `BoardGame`: posizionare una carta, attaccare l'eroe avversario oppure attaccare una creatura. Per farlo invoca rispettivamente `place`, `attackHero` e `attackCard`. Il `MatchController` rimane cosi' il coordinatore del turno, ma non contiene logica decisionale ne' dettagli di conversione tra azioni simulate e azioni reali.

2.2.2 Simone Marsili

Problema 1

Problema:

Soluzione:

Problema 2

Problema:

Soluzione:

Problema 3

Problema:

Soluzione:

Problema 4

Problema:

Soluzione:

2.2.3 Francesco Dama

Problema 1

Problema:

Soluzione:

Problema 2

Problema:

Soluzione:

Problema 3

Problema:

Soluzione:

Problema 4

Problema:

Soluzione:

2.2.4 Lorenzo Mercuriali

Problema 1

Problema:

Soluzione:

Problema 2

Problema:

Soluzione:

Problema 3

Problema:

Soluzione:

Problema 4

Problema:

Soluzione:

3. Sviluppo

3.1 Testing automatizzato

Il testing automatizzato e' stato realizzato con JUnit ed eseguito tramite il task Gradle `test`, configurato con `useJUnitPlatform()`. I test sono concentrati sulla parte di modello dell'applicazione. In particolare sono stati testati i componenti che gestiscono giocatore, mano, mazzo, armata, creature e caricamento delle definizioni delle carte. Sono inoltre presenti test per il `BoardGame` e per le componenti dell'intelligenza artificiale, come generazione delle azioni legali, transizioni di stato simulato, valutazione euristica e scelta delle mosse. In questo modo le principali regole della partita e il comportamento automatico dell'avversario possono essere controllati in modo ripetibile a ogni esecuzione della suite.

3.2 Note di sviluppo

3.2.1 Massimiliano Ria

3.2.2 Simone Marsili

3.2.3 Francesco Dama

3.2.4 Lorenzo Mercuriali

4. Commenti Finali

4.1 Autovalutazione e lavori futuri

4.1.1 Massimiliano Ria

4.1.2 Simone Marsili

4.1.3 Francesco Dama

4.1.4 Lorenzo Mercuriali

4.2 Opzionale - Difficoltà incontrate e commenti per i docenti

Appendice A

Guida Utente

Appendice B

Esercitazioni di Laboratorio